

第六章 设备管理

- 第一节 I/O系统的功能、模型和接口
- 第二节 I/O设备和控制方式
- 第三节 中断机构和中断处理程序
- 第四节 设备驱动程序
- 第五节 与设备无关的IO软件
- 第六节 用户层的IO软件
- 第七节 缓冲区管理
- 第八节 磁盘存储器的性能和调度

2025/12/8/Mon

1

6.1 I/O系统的功能、模型和接口

I/O系统管理的对象：I/O设备和相应的设备管理器。

任务：完成用户提出的I/O请求，提高I/O速率，提高设备利用率。

- I/O系统的功能
- I/O系统的层次结构和模型
- I/O系统的接口

6.1.1、I/O系统的基本功能

- 1、隐藏物理设备的细节

I/O设备类型多，差异大。利用设备控制器（硬件）进行控制。
隐藏物理设备的使用细节。

- 2、与设备的无关性

用户仅提供逻辑设备名来使用设备；即插即用功能。

- 3、提高处理机和I/O设备的利用率

设备之间、设备与处理机之间均可并行操作。要求CPU快速响应I/O请求，减少对设备运行的干预时间。

- 4、对I/O设备进行控制

即驱动程序的功能。具体控制方式与设备的传输速率和传输数据单位有关。

- 5、确保对设备的正确共享

以共享属性来分类，分为独占设备、共享设备。

- 6、错误处理

设备包括机械和电气部分，易出错和故障。临时性错误（重试）、持久性错误（向上层报告）

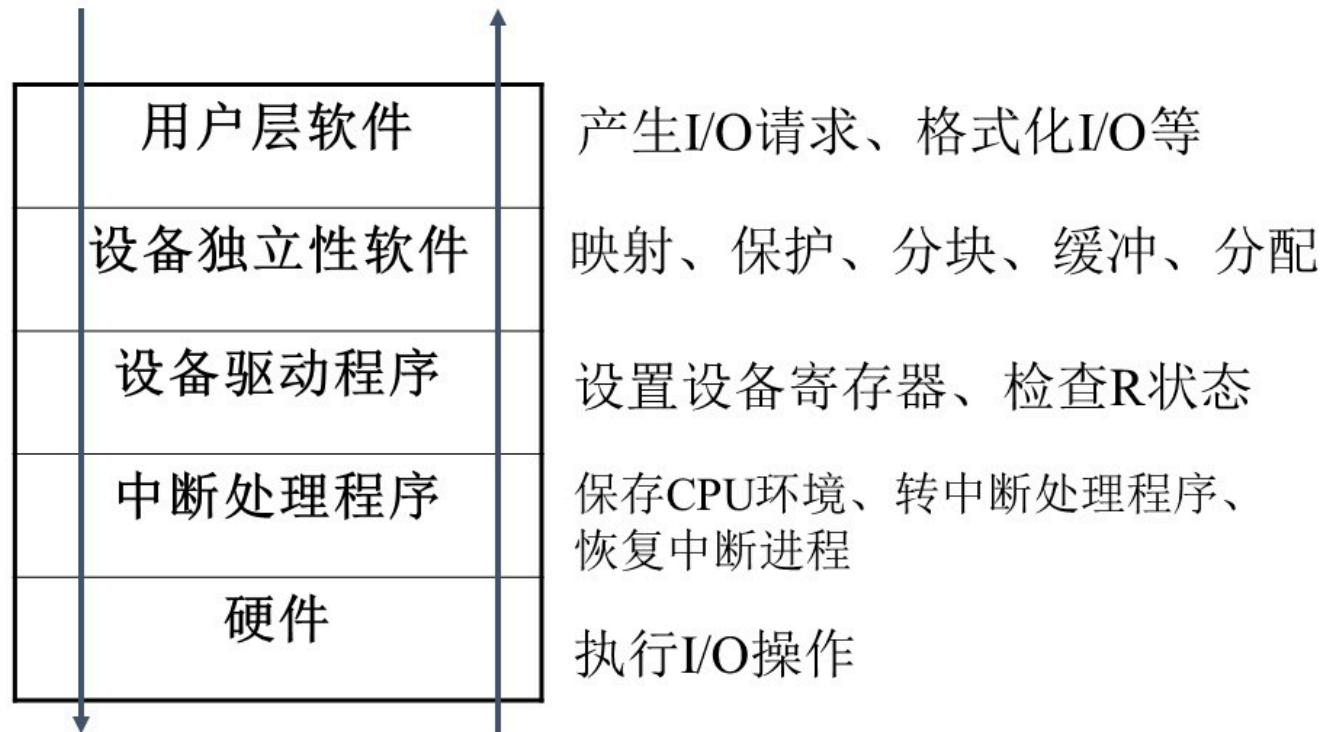
6.1.2、I/O系统的层次和模型

I/O软件向下与硬件相关，向上与文件系统、虚拟存储、用户直接交换，都需要 I/O系统来实现 I/O操作。目前均为层次式 I/O系统设计，单向调用。

● 1、I/O软件的层次——四层

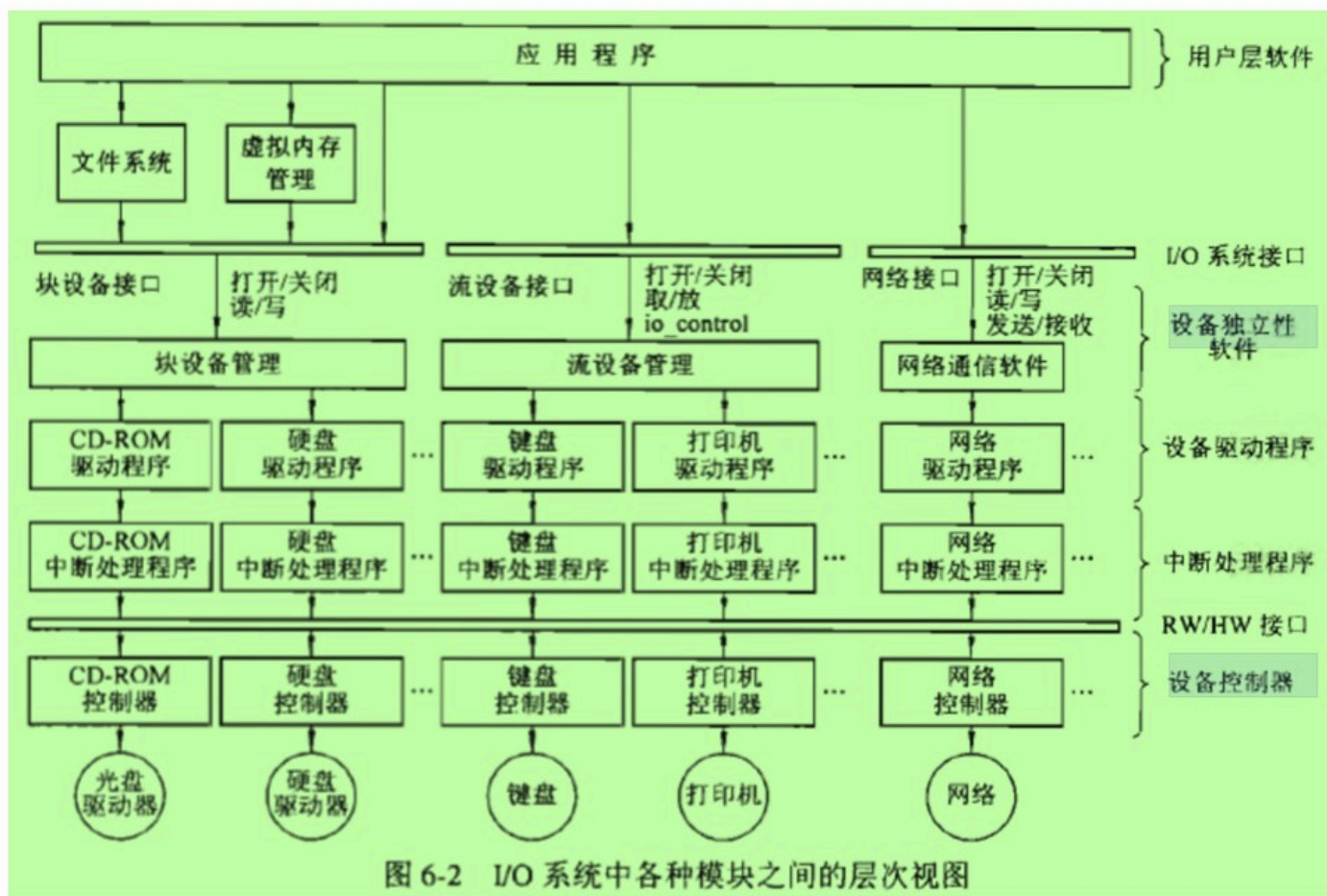
- 1) 用户层软件：实现与用户交互的接口
- 2) 设备独立性软件：
- 3) 设备驱动程序：实现系统对设备发出指令
- 4) 中断处理程序

I/O应答



2025/12/8/Mon

6



2025/12/8/Mon

7

- 2、I/O系统中各种模块之间的层次试图（图6.2）

- 1) I/O系统的上、下接口

- (1) I/O系统接口

是I/O系统与上层系统之间的接口。向上层提供对设备操作的抽象命令，供高层对设备的使用。OS在用户层中提供了与I/O操作有关的库函数，供用户使用。

- (2) 软件/硬件接口

在上下两个接口之间则是I/O系统。

2) I/O系统的分层

(1) 中断处理程序

当有I/O设备发来中断请求信号时，先中断硬件初步处理，后转中断处理程序。

(2) 设备驱动程序

进程和设备控制器之间的通信程序。将上层发来的抽象请求转化为对设备的具体参数和命令，装入控制器中。

(3) 设备独立性软件

设备无关性软件。I/O软件独立于具体使用的物理设备。

6.1.3、I/O系统接口

I/O系统与高层之间的接口中，根据设备类型不同，分为块设备接口、流设备接口、网络接口。

- 1、块设备接口——控制磁盘、光盘设备的输入输出
 - (1) 块设备：磁盘等。可寻址。DMA控制方式。
 - (2) 隐藏了磁盘的二维结构。扇区编号0——n-1
 - (3) 将上层的抽象命令映射为低层操作。如将抽象命令的逻辑块号转化为盘面、磁道、扇区。

● 2、流设备接口—控制字符设备的输入输出

- (1) 字符设备：键盘、打印机等。不可寻址。中断驱动控制方式。
- (2) put和get操作。为字符设备建立一个字符缓冲区。
- (3) in-control指令。统一处理各类字符设备。包含许多参数，每个参数对应一个具体设备。

● 3、网络通信接口

操作系统需提供相应的网络软件和网络通信接口

6.2 I/O设备和设备控制器

I/O设备有机械部分和电子部分组成。机械部分即为一般的I/O设备，电子部分称为设备控制器或适配器（控制卡、接口卡、网卡）

- I/O设备
- 设备控制器
- I/O通道
- 总线系统

6.2.1、I/O设备

- 1、I/O设备的类型

- 按传输速率分类：低速、中速、高速设备
- 按信息交换单位分类：块设备、字符设备
- 按设备的共享属性：独占、共享、虚拟设备

- 2、设备与控制器之间的接口

三种类型的信号线：

- 数据信号线：用于设备和控制器之间传送数据信号
- 控制信号线：控制器向设备发送控制信号时的通路
- 状态信号线：传送指示设备当前状态的信号

6.2.2、设备控制器

- 一些设备会有机械和电子两部分。电子部分称为设备控制器。它是CPU和设备之间的接口。
- 设备控制器负责控制一个或多个I/O设备，实现设备和计算机间的数据交换。
- 基本功能：
 - 1、接收和识别命令
在控制器中应具有控制寄存器，用来存放接收的命令和参数，并进行译码

- 基本功能（续上）：

- 2、数据交换——寄存器

实现CPU与控制器，控制器与设备间的数据交换

- 3、标识和报告设备的状态

控制器中的状态寄存器记录设备的状态供CPU了解

- 4、地址识别——地址译码器

每个设备都有一个地址，控制器必须能识别。

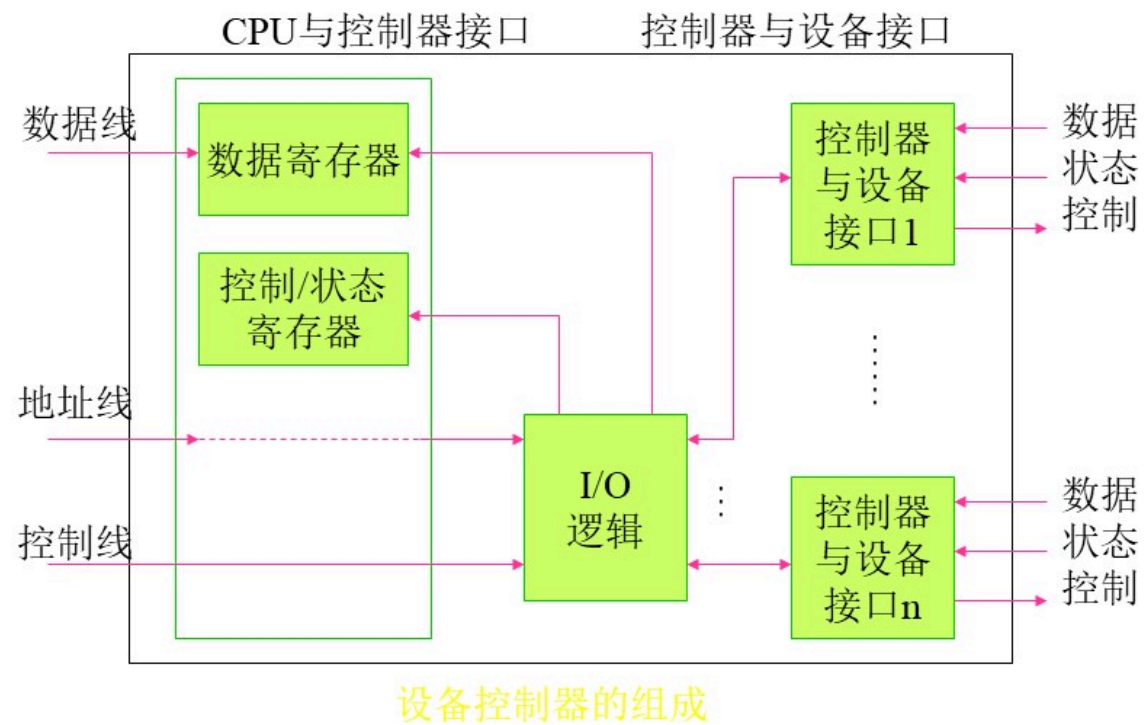
- 5、数据缓冲——缓冲器

解决I/O设备与CPU、内存速度不匹配的矛盾

- 6、差错控制

控制器兼管对由I/O设备传送来的数据进行差错检测

- 设备控制器的组成：
 - 设备控制器与处理机的接口
共有三类线：数据线、地址线、控制线
 - 设备控制器与设备的接口
每个接口中都存在数据、控制、状态三种信号
 - I/O逻辑——实现对设备的控制
CPU利用它向控制器发送命令；
负责对命令译码和地址译码。



6.2.3 内存映像I/O

驱动程序将IO命令转化为具体值装入设备控制器的寄存器，可用如下方法完成：

- 1、利用特定的I/O指令

如，将CPU寄存器的内容存入内存的某个单元（k）中：

Store cpu-reg , k

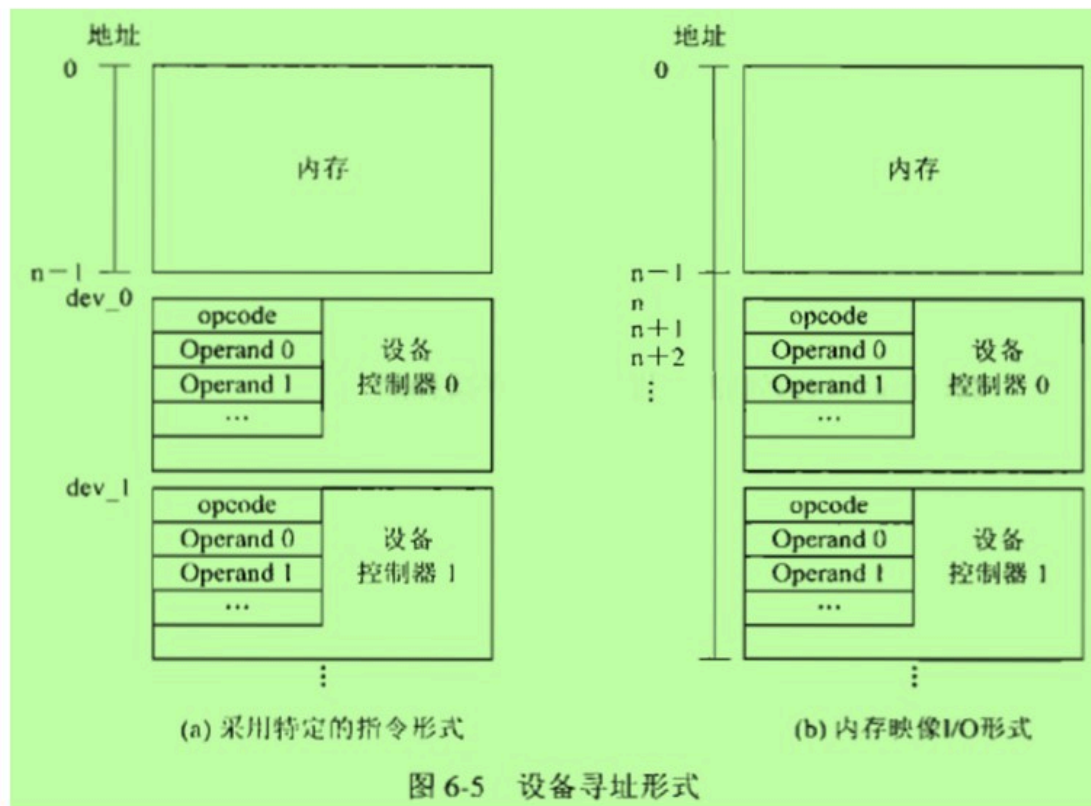
访问设备： io-Store cpu-reg, dev-no, dev-reg

缺点：访问内存和访问设备需要两种不同的指令。

- 2、内存映像I/O

不区分内存单元地址和设备控制器的寄存器地址。k处于0—n-1，内存地址；k≥n，控制器的寄存器地址。

6.2.3 内存映像I/O



2025/12/8/Mon

19

6.2.4、I/O通道

- 通道

- 专门用于处理I/O的处理机，控制内存和外设直接进行数据交换。
- 目的：减轻CPU的负担
- 工作方式：CPU向通道发送I/O命令，通道执行通道程序，完成I/O后向CPU发送中断信号。

- 通道与一般处理机的不同：

- 指令类型单一，仅能执行I/O指令。
- 通道没有自己的内存，通道程序放在主存中。

- 通道的类型

- 字节多路通道:

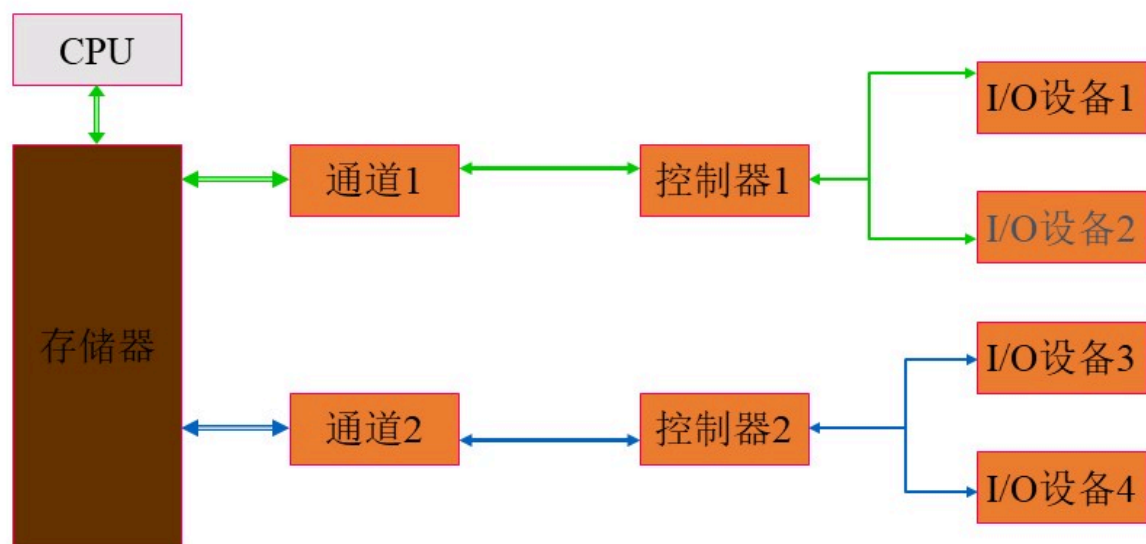
是一种按字节交叉方式工作的通道，采用多路分时复用——按时间片轮转方式共享主通道。

- 数组选择通道: 独占使用，成组（块）传送；

- 数组多路通道: <上述两种技术的结合>

●“瓶颈”问题

- 通道资源有限，系统需要同时启动的设备可能较多

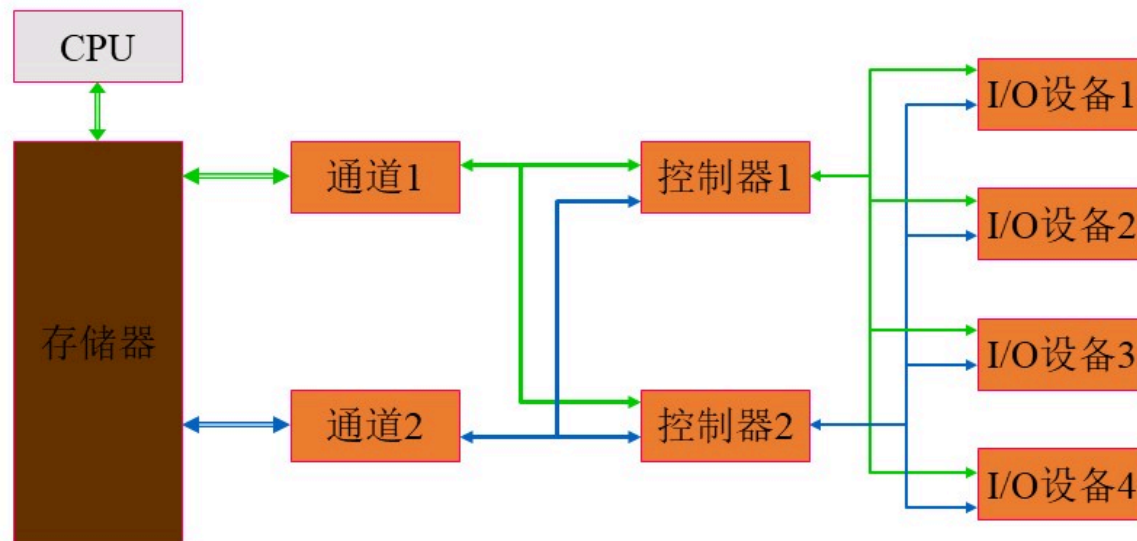


2025/12/8/Mon

22

●“瓶颈”问题

- 解决方法：增加设备到主机间的通路。



2025/12/8/Mon

23

6.3 中断机构和中断处理程序

中断是多道程序实现的基础（进程切换），也是设备管理的基础（处理机和外设并行执行）。中断是IO系统最低一层，是整个IO系统的基础。

6.3.1 中断简介

1、中断和陷入

（1）中断——CPU对IO设备发来的中断信号的一种响应。中断是由外部设备引起，也称为外中断。

（2）陷入——由CPU内部事件引起的中断。如溢出、非法指令、地址越界、电源故障等。也称为内中断。

2、中断向量表和中断优先级

1) 中断向量表

为每种设备配以相应的中断处理程序，把该程序的入口地址放在中断向量表的一个表项中，并规定一个中断号用于设备的中断请求。

2) 中断优先级

系统中有多个中断信号源，系统为他们规定不同的优先级。如：键盘<打印机<磁盘

3、对多中断源的处理方式（处理打印中断的时收到磁盘中断信号）

1) 屏蔽（禁止）中断

对任何新到的中断请求都暂时不处理，让其等待。

2) 嵌套中断

多中断请求时，优先响应优先级最高的；高优先级中断请求可抢占低优先级中断的处理机。

6.3.2 中断处理程序

当一个进程请求I/O操作时，该进程将被挂起。直到设备完成I/O操作后，设备控制器向CPU发送一个中断请求。CPU响应后，转中断处理程序。中断处理程序执行相应的处理，处理完后解除进程的阻塞状态。

中断处理程序的处理过程分成以下几个步骤：

(1) 测定是否有未响应的中断信号

当设备完成一个字符的读入（如字符设备），设备控制器向处理机发送一个中断请求信号，请求处理机将字符读入内存。处理机执行完当前指令后都要检测是否有未响应的中断信号。

(2) 保护被中断进程的CPU环境

保存程序状态字PSW和程序计数器PC中下一条指令的地址，入中断保留区（栈）中。所有CPU寄存器的内容入栈。

(3) 转入相应的设备处理程序

处理机测试各个中断源，确定IO设备，并发送确认信号。设备收到信号后，取消中断请求信号。将设备中断处理程序的入口地址装入到PC中。

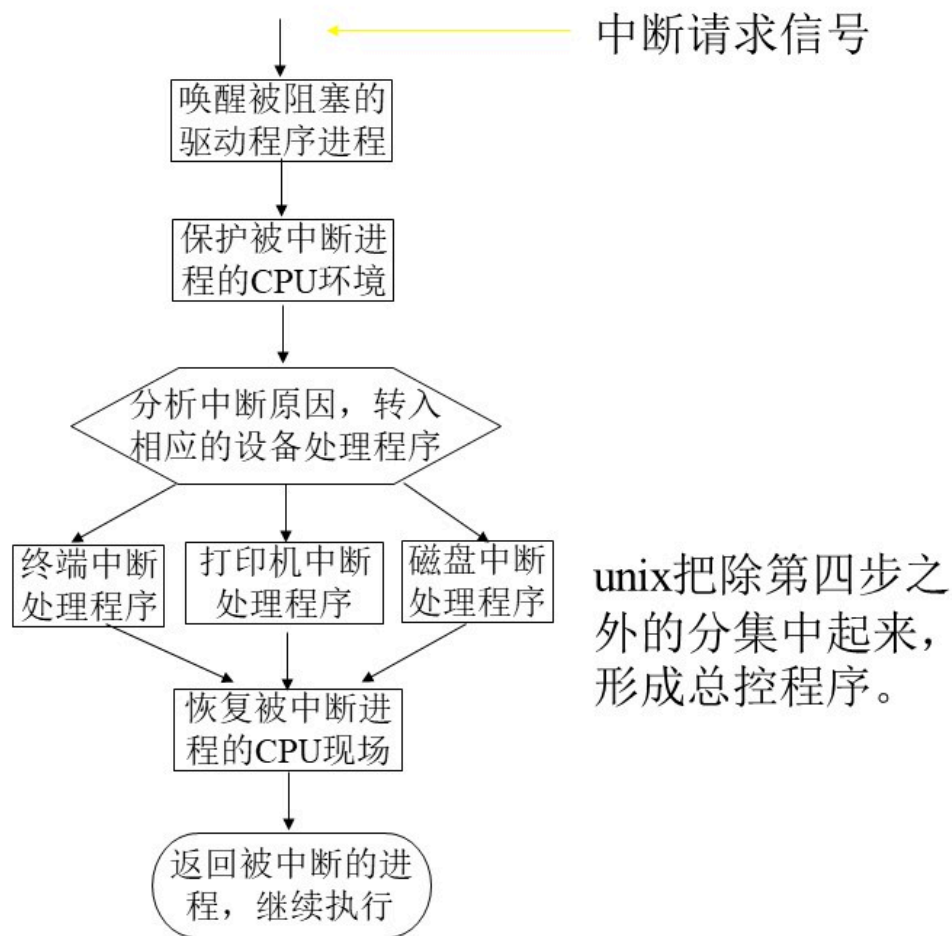
(4) 进行中断处理

不同的设备有不同的中断处理程序。

(5) 恢复被中断进程的现场

中断处理完成后，恢复CPU现场，退出中断。是否返回被中断的进程，取决于：

- a) 采用屏蔽中断方式。返回至被中断的进程
- b) 采用中断嵌套方式，考虑有无优先级更高的中断请求



2025/12/8/Mon

29

6.4 设备驱动程序

设备处理程序又称设备驱动程序，是I/O进程与设备控制器之间的通信程序。

主要任务：

接收上层软件发来的抽象要求，比如：Read、Write等命令，再把它们转化为具体要求发送给设备控制器，启动设备执行；同时，还负责把设备控制器发来的信号传送给上层软件。

6.4.1 设备驱动程序概述

● 1、设备驱动程序的功能

- 接收由设备独立性软件发来的命令和参数，转换为具体要求。（盘块号->盘面、磁道号、扇区号）
- 检查用户I/O请求的合法性，了解设备状态，传递或设置参数
- 发出I/O命令，启动设备，或挂在相应的设备队列上
- 及时响应由控制器或通道发来的中断请求，根据中断类型调用相应的中断处理程序进行处理。

2025/12/8/Mon

31

● 2、设备处理方式

- 为每一类设备设置一个进程，专门用于执行该类设备的I/O操作。
- 在整个系统中设置一个I/O进程，专门用于执行系统中所有各类设备的I/O操作。
- 不设置专门的设备处理进程，只为各类设备设置相应的设备驱动程序，供用户进程或系统进程调用。
(较多采用)

●3、设备驱动程序的特点

- 在请求I/O的进程与设备控制器之间的一个通信和转换程序。
- 驱动程序与设备控制器和I/O设备的硬件特性紧密相关，不同类型的设备应配置不同的驱动程序。
- 驱动程序与I/O设备所采用的I/O控制方式紧密相关。
(中断驱动、DMA方式)
- 其中一部分必须用汇编语言书写，固化在ROM中。
- 驱动程序应允许可重入。

6.4.2 设备驱动程序的处理过程

设备驱动程序的主要任务是启动指定设备。

具体的处理过程如下：

- 将抽象要求转换为具体要求
- 检查I/O请求的合法性
- 读出和检查设备的状态
- 传送必要的参数（磁盘在读写前，要传递参数至控制器的寄存器中）
- 启动I/O设备

6.4.2 设备驱动程序的处理过程

● 1、设备驱动程序的功能

- 接收由设备独立性软件发来的命令和参数，转换为具体要求。（盘块号->盘面、磁道号、扇区号）
- 检查用户I/O请求的合法性，了解设备状态，传递或设置参数
- 发出I/O命令，启动设备，或挂在相应的设备队列上
- 及时响应由控制器或通道发来的中断请求，根据中断类型调用相应的中断处理程序进行处理。

6.4.3 对I/O设备的控制方式

- 使用轮询的可编程I/O方式
- 使用中断的可编程I/O方式
- 直接存储器访问（DMA）方式
- I/O通道控制方式

前两种是以字节为单位进行数据传递，后两种是以数据块为单位进行数据传递。

1、使用轮询的可编程I/O方式

- 忙—等待方式

- CPU向控制器发指令，启动I/O设备输入数据时，同时把状态寄存器中的状态标志busy置1；
- 然后不断地循环测试busy。
 - 如果busy=1，说明I/O设备忙，CPU再进行下一轮检测；
 - 如果busy=0，说明I/O操作结束，CPU执行下一条指令。

2、使用中断的可编程I/O方式

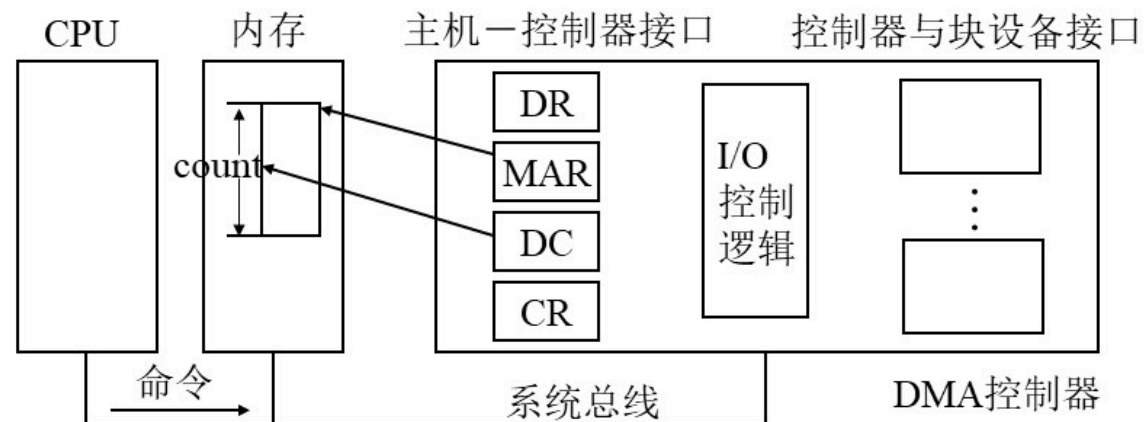
- 中断驱动I/O过程
 - 启动：由CPU根据进程的I/O请求，向设备控制器发出一条I/O命令；此后CPU继续执行其它进程，即CPU与外设并行工作。
 - I/O设备完成操作后，由控制器通过控制线向CPU发送一中断信号，由CPU检查I/O操作是否正确，.....
- 优点——系统效率高： 并行工作、用极少的时间处理中断
- 缺点： 设备与CPU之间的数据交换仍以字（节）为单位。

3、直接存储器访问 (DMA) 方式

- **1、DMA (Direct Memory Access) 控制方式的引入**
 - 适应一次传送大量数据的应用要求；
 - 尽量减少CPU对高速外设的干预；
- **该方式的特点：**
 - 数据传输的基本单位是数据块。
 - 所传送的数据是从设备直接送入内存，或者相反
 - 仅在传送一个数据块的开始和结束时，才需要CPU干预，整块数据的传送是在控制器的控制下完成的

• 2、DMA控制器的组成

- 命令/状态寄存器CR
- 内存地址寄存器MAR
- 数据寄存器DR: 暂存从设备到内存的数据, 或反之
- 数据计数器DC: 存放本次CPU要读或写的字(节)数



2025/12/8/Mon

40

• 3、DMA工作过程

(以磁盘数据读入为例)

- CPU向磁盘控制器发送:读命令到CR中、内存的目标起址到MAR中、数据字节数到DC中、磁盘中的源地址到控制器的控制逻辑上;
- 启动DMA控制器进行数据传送。此后, CPU可以执行其它任务;
- DMA控制器按照命令传送数据: 先从磁盘读入一个字节的数送入DR后, 再传送到内存中。
- 修改并检查DC中的数值: 若DC中的值不为0, 则继续传送下一个字节; 为0, 则发出一个中断请求。

2025/12/8/Mon

41

4、I/O通道控制方式

- 1、I/O通道控制方式的引入

- 进一步减少CPU对I/O操作的干预；
- 以多个块为单位进行数据传送；
- 一次传送多组数据到多个不同的内存区域。

- 2、通道程序

- 通道是通过执行通道程序，并与设备控制器共同实现对I/O设备的控制的。
- 通道程序由一系列通道指令（通道命令）构成。

每条指令都包含以下信息：

- 1、操作码——指令执行的操作：读/写
- 2、内存地址——字符送入/取出内存的首址
- 3、计数——表示本条指令所要读/写数据的字节数
- 4、通道程序结束位P P=1表示本条指令是最后一条
- 5、记录结束标志位R R=1表示这是处理某记录的最后一条指令

下列通道程序的功能：

将内存中不同地址的数据，写成多个记录。

操作	P	R	计数	内存地址
write	0	0	80	813
write	0	0	140	1034
write	0	1	60	5830
write	0	1	300	2000
write	0	0	250	1850
write	1	1	250	720

2025/12/8/Mon

44

6.5 与设备无关的I/O软件

- 设备独立性（设备无关性）的含义：

应用程序中所用的设备，不局限于使用某个具体的物理设备。

- 与设备无关的I/O软件（设备独立性软件）含义：

在设备驱动程序之上设置一层软件，以实现设备独立性。

6.5.1与设备无关软件的基本概念

- 1、以物理设备名使用设备

应用程序中所用的设备与系统中的物理设备直接相关。

问题：不灵活，用户不方便。

- 2、引入了逻辑设备名

逻辑设备是抽象的设备名，如/dev/printer，并不指定哪一台打印机。好处：系统设备分配灵活。

可实现I/O重定向。即：用于IO操作的设备可更换，而不必改变应用程序。

如修改逻辑设备表：终端->打印机

- 3、逻辑设备名到物理设备名的映射
 - 逻辑设备表 (Logical Unit Table)

实现逻辑设备名到物理设备名的映射，将应用程序中所使用的逻辑设备名映射为物理设备名，并提供该设备驱动程序的入口地址。

逻辑设备名	物理设备名	驱动程序入口地址
/dev/tty	3	1024
/dev/printer	5	2046
...	...	...

2、逻辑设备表的设置问题

(1) 单用户系统中，在整个系统中设置一张LUT，不允许有相同的逻辑设备名

(2) 多用户系统中，为每个用户设置一张LUT，用户登录时，建立LUT并放入进程的PCB中。

6.5.2 与设备无关的软件

设备无关的软件是I/O系统的最高层软件，包括了执行所有设备公有操作的软件。具体包括：

- 1、设备驱动程序的统一接口

可以方便添加新设备的驱动程序。

- 2、缓冲管理

- 3、差错控制——暂时性错误、永久性错误

- 4、对独占设备的分配与回收

对独占设备由系统进行统一的分配和回收处理。

- 5、独立于设备的逻辑数据块

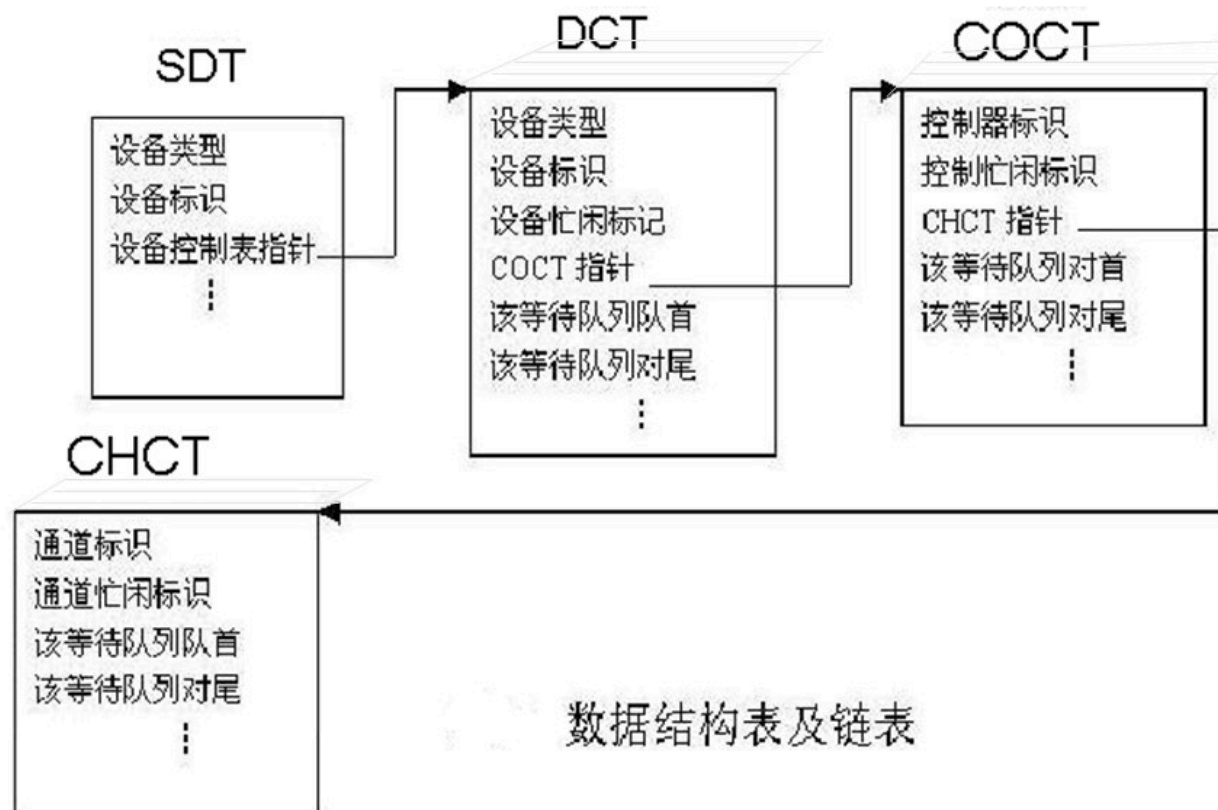
隐藏不同设备的物理数据块大小的差异，向高层软件提供大小统一的逻辑数据块。

6.5.3 设备分配

1、设备分配中的数据结构

为了实现对I/O设备的管理和控制，需要对每台设备、通道、控制器的情况进行登记。设备分配程序依据这些信息进行设备分配。

- 设备控制表DCT（为每台设备配置一张）
- 控制器控制表COCT
- 通道控制表CHCT
- 系统设备表SDT



数据结构表及链表

2、设备分配时应考虑的因素

- 1、设备的固有属性
 - 独享设备
 - 共享设备
 - 虚拟设备
- 2、设备分配算法
 - 先来先服务
 - 优先级高者优先
- 3、设备分配中的安全性
 - 安全分配方式：进程发出I/O请求后就进入阻塞态
 - 不安全分配方式：进程发出I/O请求仍继续运行

3、独占设备的分配程序

1、基本的设备分配程序

分配设备→分配控制器→分配通道

2、设备分配程序的改进

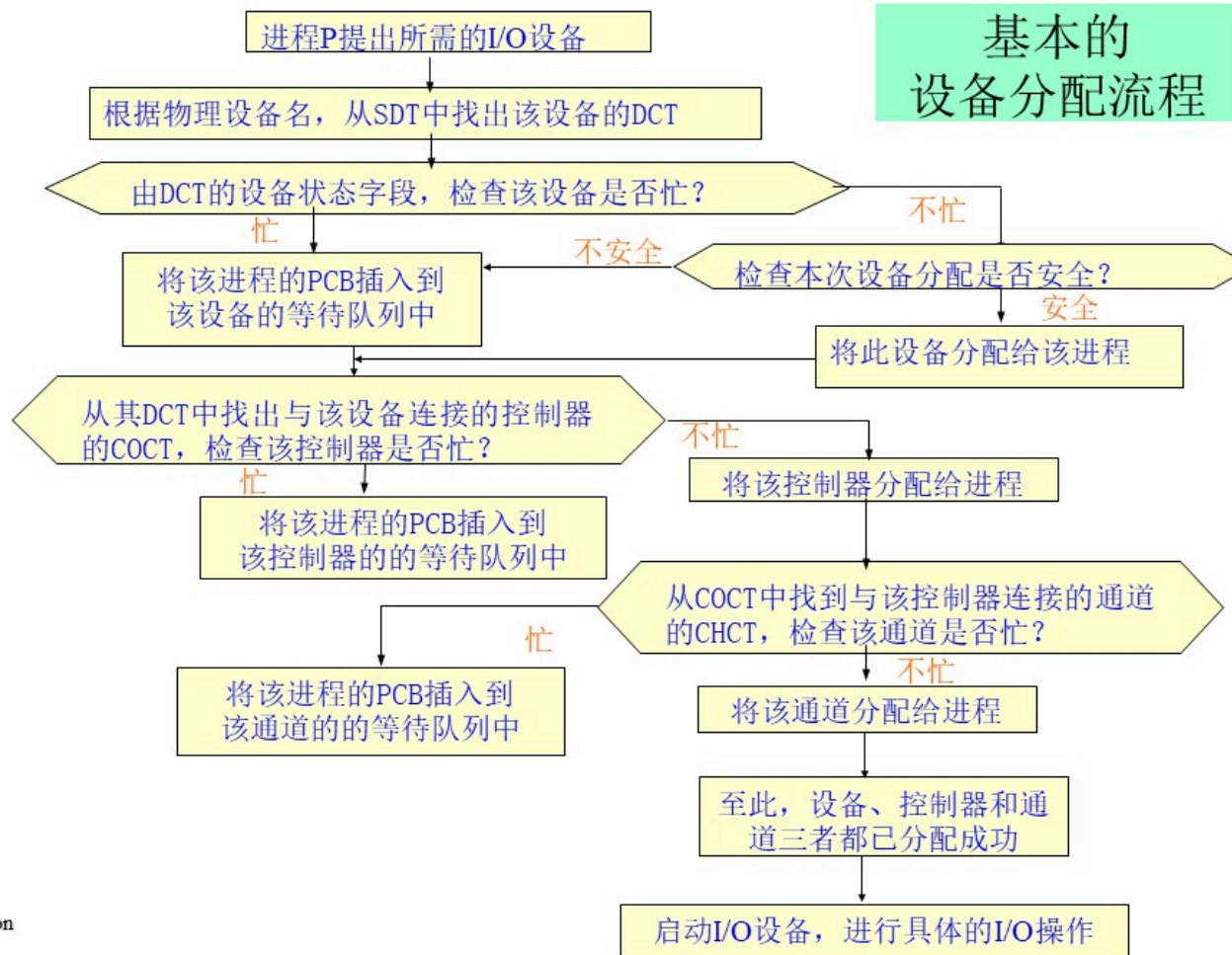
1) 增加设备的独立性

逻辑设备名请求I/O。系统应查找所有该类设备的DCT。

2) 考虑多通路情况

对多个控制器和通道的分配，必须查找所有的控制器和通道。

基本的 设备分配流程



2025/12/8/Mon

54

6.6 用户层的I/O软件

大部分的IO软件都放在操作系统内部。小部分在用户层。其中包括与用户程序链接在一起的库函数，及完全运行于内核之外的假脱机系统等。

- 系统调用
- 库函数
- 假脱机系统

6.6.1 系统调用与库函数（自学）

- 1、系统调用
- 2、库函数

2025/12/8/Mon

56

6.6.2 假脱机 (SPOOLing) 系统

- 1、什么是SPOOLing技术（假脱机技术）
 - 定义：在联机的情况下实现的同时外围操作
 - 特点：是对脱机输入输出系统的模拟。因此，必须建立在具有多道程序功能的操作系统上，而且需要高速外存的支持。
 - 方式：将数据从输入设备传送到磁盘或反之。
 - 通过它可以将一台独占的物理设备虚拟为多台逻辑设备，从而允许多个用户（进程）共享。

2、SPOOLing系统的组成

- 输入井和输出井：

在磁盘上开辟的两个大存储空间

- 输入缓冲区和输出缓冲区：

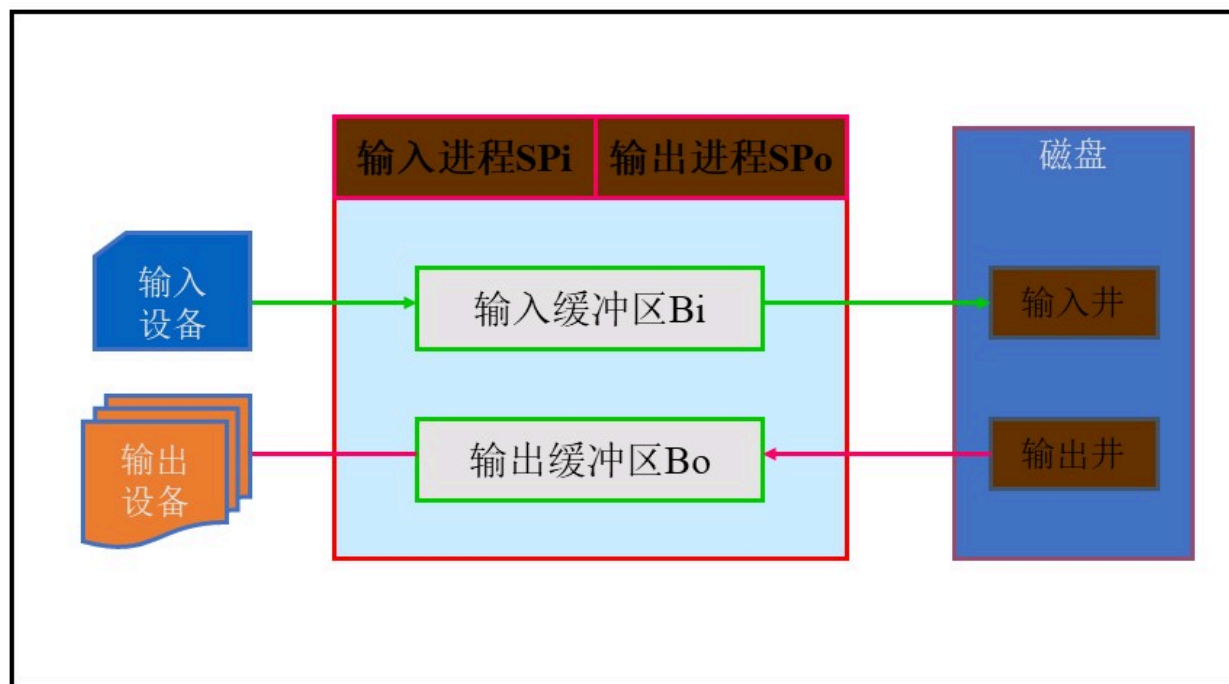
在内存中开辟的两个缓冲区

- 输入进程 Sp_i 和输出进程 Sp_o ：

是内存中的两个进程

- 井管理程序：

控制作业与磁盘井之间信息的交换



2025/12/8/Mon

59

- 3、假脱机打印机系统

- SPOOLing系统接收到用户进程的打印输出请求后：
 - 由输出进程在输出井中为之申请一个空闲磁盘块区，并将要打印的数据送入其中；
 - 输出进程再为用户进程申请一张空白的用户请求打印表，并将用户的打印要求填入其中，再将该表挂到请求打印队列上。
- 当打印机空闲时，输出进程将从请求打印队列的队首取出一张请求打印表，再从输出井把数据送到内存缓冲区，启动打印机打印输出。打印完后，输出进程检查请求打印队列.....

●4、SPOOLing系统的特点:

- 提高了I/O的速度，缓和了CPU与低速I/O设备速度不匹配的矛盾
- 利用高速共享设备，将独占设备改造为共享设备
- 实现了虚拟设备功能：用户都感到独占了一台设备

●5、守护进程（daemon）

- 守护进程取代假脱机管理进程。执行以下功能：

为用户申请磁盘缓冲区的空闲盘块、写入打印数据、返回该盘块的首址。

- 守护进程是允许使用打印机的唯一进程。打印进程要将要求打印的文件放在假脱机文件队列中。并唤醒守护进程。

- 守护进程的任务是按照队列中每个文件的说明执行逐个打印任务，直至全部完成，然后休眠。

- 总之，凡是需要将独占设备改造为共享设备时，都要为该设备配置守护进程和假脱机文件队列。

6.7 缓冲区管理

- 缓冲的引入
- 单缓冲和双缓冲
- 循环缓冲
- 缓冲池

2025/12/8/Mon

63

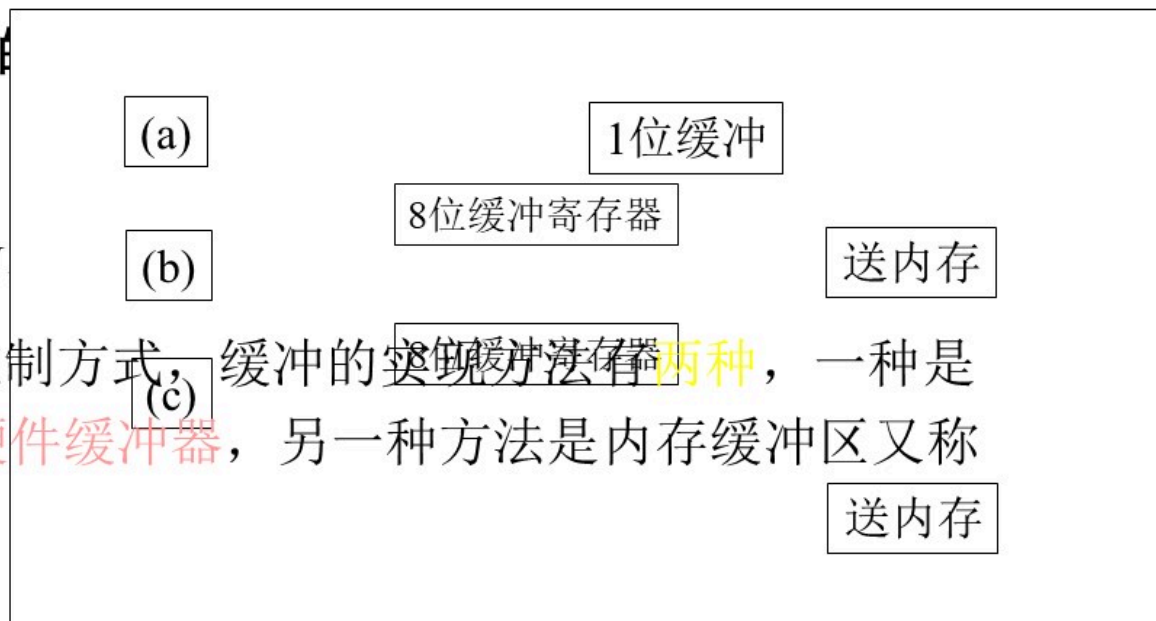
6.7.1、缓冲的引入

- 缓和CPU与I/O设备间速度不匹配的矛盾

- 减少对CPU时间的限制

- 提高CPU

- 根据I/O控制方式，缓冲的实现方法有**两种**，一种是采用**专用硬件缓冲器**，另一种方法是内存缓冲区又称**软件缓冲**。



根据系统设置的缓冲器的个数，可把
缓冲技术分为：

单缓冲

双缓冲

循环缓冲

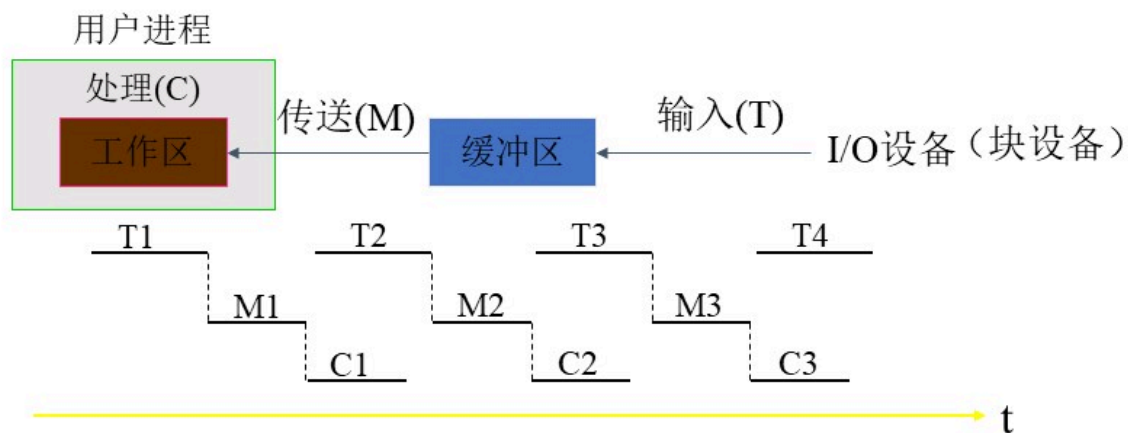
缓冲池

• 6.7.2 单缓冲和双缓冲

• 1、单缓冲

- 在设备和处理机之间设置一个缓冲区。
- 输入T和处理C是可以并行的。系统对每个数据的处理时间为 $\text{Max}(C, T) + M$ 。

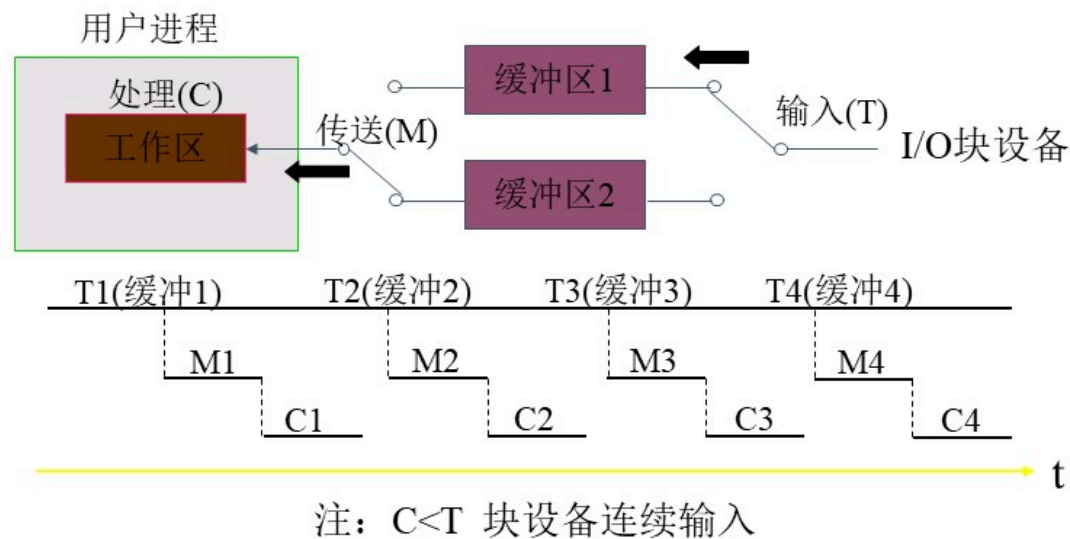
以块设备数据读入为例：



●2、双缓冲—缓冲对换

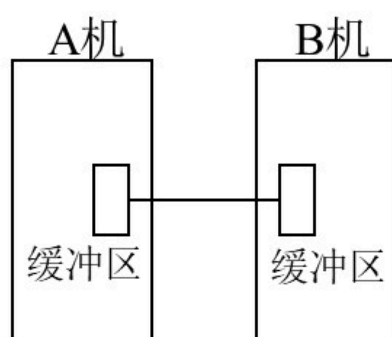
- 目的：加快输入输出的速度。
- 系统处理每个数据的时间可粗略认为 $\text{Max}(C, T)$ 。当 $T > C$ ，可使块设备连续输入；反之可使CPU不必等待设备输入

以块设备数据读入为例：

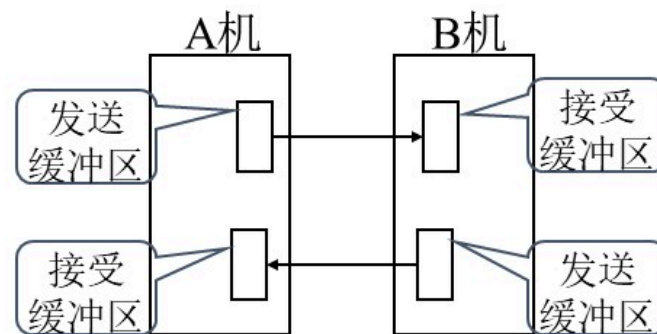


如果在实现两台机器通信时，只配置单缓冲，那么在任一时刻只能实现单方向的数据传输。

为了实现双向数据传输，必须在两台机器中都设置两个缓冲区。分别用在发送和接收。



(a)单缓冲



(b)双缓冲

6.7.3 循环缓冲

- 循环缓冲的引入

- 当输入和输出的速度相差很大时，双缓冲效果不理想，但可增加缓冲区的数量，改善情况

循环缓冲是把多个缓冲区连接起来组成两部分，一部分专门用于输入，另一部分专门用于输出的缓冲结构。

例如对于用做输入的循环缓冲区，通常提供给输入进程和计算进程使用，输入进程不断向空缓冲区中输入数据，计算进程则从满缓冲区中提取数据用于计算。

●1、循环缓冲的组成：

- 多个缓冲区、多个指针

- 每个缓冲区大小相同。

- 用做输入的缓冲区的类型有：

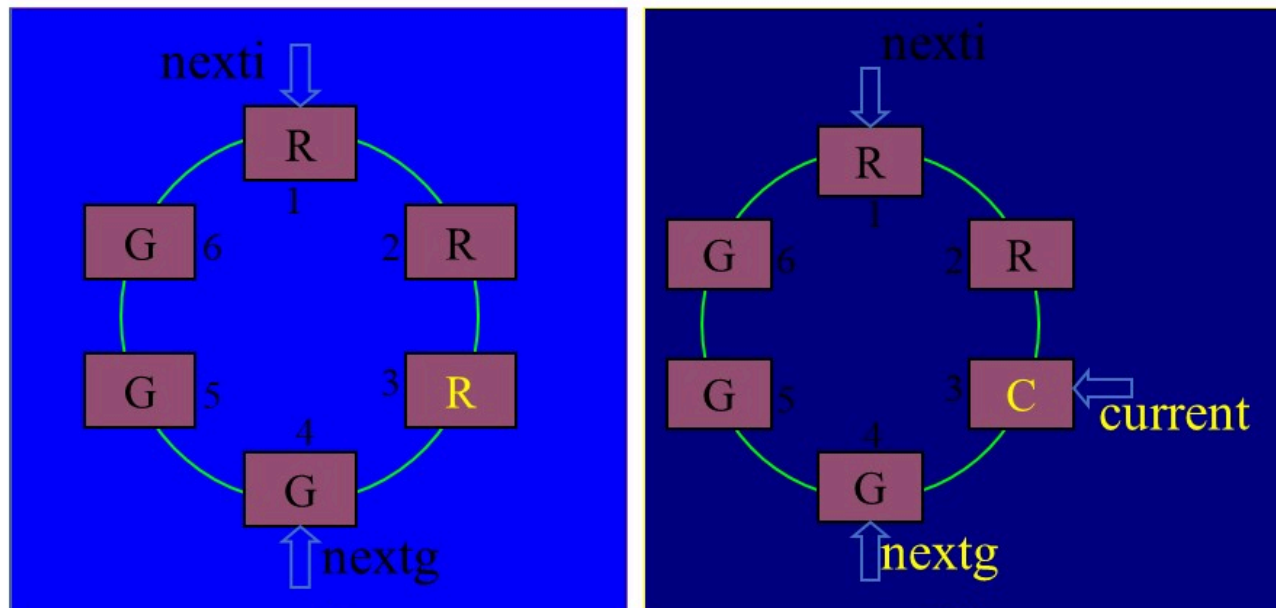
空缓冲区R、已装满数据的缓冲区G、工作缓冲区C

- 指针类型：

指示计算进程的下一个可用缓冲区G的指针nextg

指示输入进程下次可用的空缓冲区R的指针nexti

指示计算进程正在使用的缓冲区C的指针current



● 计算进程和输入进程可利用下述两个过程使用循环缓冲区：

● **GetBuf()** **ReleaseBuf()**

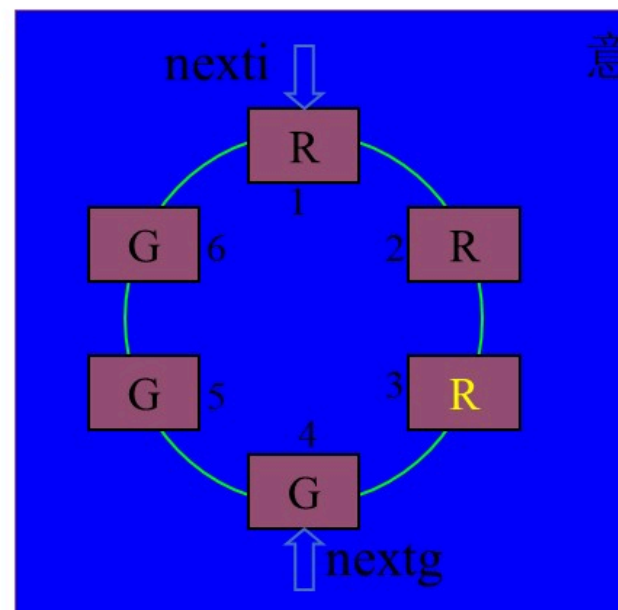
●3、进程的同步

- Nexti指针追上Nextg指针。

意味着输入进程输入数据的速度大于计算进程处理数据的速度，再无空缓冲区可用。输入进程应阻塞。

- Nextg指针追上Nexti指针。

意味着输入数据的速度低于计算进程处理数据的速度，再无装有数据的缓冲区可用。计算进程应阻塞。



2025/12/8/Mon

72

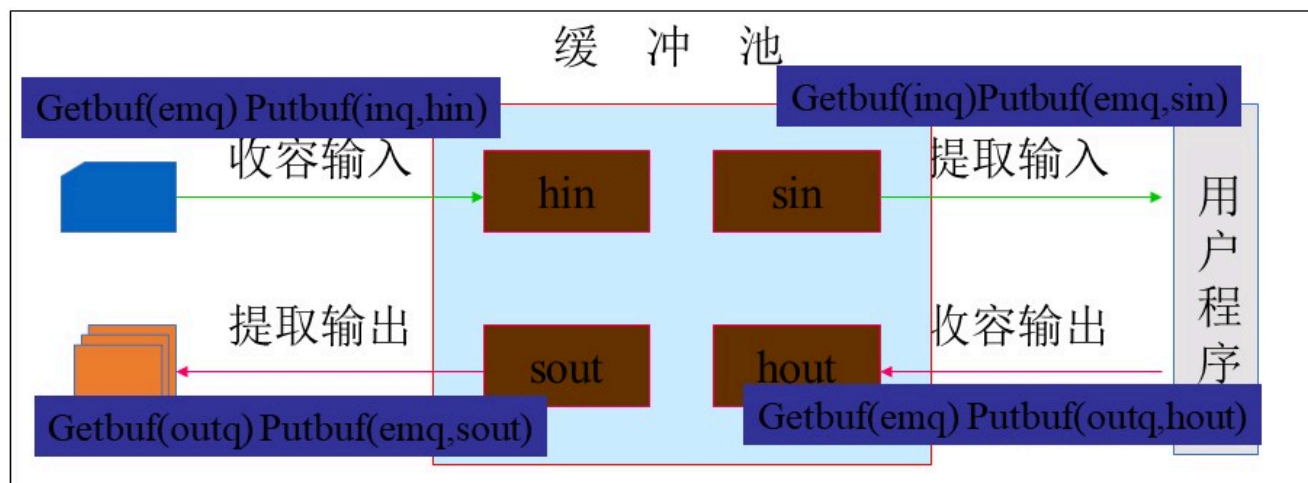
6.7.4 缓冲池

- 缓冲池的引入
 - 把专用循环缓冲变为公用缓冲池提高内存利用率。
- 同时用于输入/输出的公用缓冲池的组成：
 - 空闲缓冲区
 - 装满输入数据的缓冲区
 - 装满输出数据的缓冲区
- 组织队列：
 - 空缓冲（区）队列emq：
 - 队首指针F(emq)、队尾指针L(emq)。

- 输入缓冲（区）队列inq：
队首指针F(inq)、队尾指针L(inq)
- 输出缓冲（区）队列outq：
队首指针F(outq)、队尾指针L(outq)
- 由于缓冲区是临界资源，在使用缓冲区时都有申请、释放和互斥的问题。
- 除了上述三个队列外，还有四种工作缓冲区：
 - 用于收容输入数据的工作缓冲区
 - 用于提取输入数据的工作缓冲区
 - 用于收容输出数据的工作缓冲区
 - 用于提取输出数据的工作缓冲区

●缓冲区的四种工作方式

- 收容输入：收容输入设备的输入数据
- 提取输入：计算进程提取缓冲区中的数据使用
- 收容输出：计算进程输出结果数据到缓冲区
- 提取输出：输出设备提取缓冲区中的数据



2025/12/8/Mon

75

6.8 磁盘存储器的性能和调度

- 磁盘性能简述
- 磁盘调度
- 磁盘高速缓存

2025/12/8/Mon

76

6.8.1、磁盘性能简述

- 1、数据的组织和格式
 - 盘片和盘面，磁道（500-2000），扇区（10-100）
 - 格式化
 - 每个扇区包括两个字段：标识符字段、数据字段
- 2、磁盘的类型
 - 固定头磁盘：每条磁道上都有磁头，装在磁臂中。
 - 移动头磁盘：每个盘面一个磁头，装在磁臂中，磁头能够移动以寻道。

• 3、磁盘访问时间

$$T_a = T_s + \frac{1}{2r} + \frac{b}{rN}$$

• 寻道时间 T_s

- 把磁臂（磁头）移动到指定磁道上所经历的时间，包含启动磁臂和磁头移动n条磁道所花费的时间。 $T_s = m \times n + s$

• 旋转延迟时间 T_r

- 指定扇区移动到磁头下面所经历的时间。与盘面的旋转速度有关。
- 5400转—平均旋转延迟时间5.55ms;
- 7200转—平均旋转延迟时间4.16ms

• 传输时间 T_t

- 把数据从磁盘读出或向磁盘写入数据所经历的时间。与旋转速度和一次读写的数据量有关。 $T_t = \frac{b}{rN}$

6.8.2 磁盘调度

P₂₁₇

- 先来先服务FCFS
- 最短寻道时间优先SSTF
- 扫描 (SCAN) 算法
- 循环扫描 (CSCAN) 算法
- N步SCAN算法
- FSCAN算法

2025/12/8/Mon

79

1、先来先服务FCFS

- 根据进程请求访问磁盘的先后次序进行调度。
- 优点：
 - 公平、简单，每个进程的请求依次得到处理
- 缺点：
 - 平均寻道时间可能较长，仅适用于磁盘请求较少的场合。

有九个进程先后提出
磁盘I/O请求：
55, 58, 39, 18, 90,
160, 150, 38, 184

(从100号磁道开始)	
被访问的下一个磁道号	移动距离 (磁道数)
55	45
58	3
39	19
18	21
90	72
160	70
150	10
38	112
184	146
平均寻道长度: 55.3	

FCFS调度算法

2025/12/8/Mon

81

2、最短寻道时间优先 (SSTF)

- 选择要求访问的磁道与当前磁头所在的磁道距离最近的进程（磁盘请求），使每次的寻道时间最短。
- 该算法不能保证平均寻道时间最短。
- 可能导致“饥饿”现象。

有九个进程先后提出
磁盘I/O请求：
55, 58, 39, 18, 90,
160, 150, 38, 184

(从100号磁道开始)	
被访问的下一个磁道号	移动距离 (磁道数)
90	10
58	32
55	3
39	16
38	1
18	20
150	132
160	10
184	24
平均寻道长度: 27.5	

SSTF调度算法

2025/12/8/Mon

83

3、扫描 (Scan) 算法

- 磁头每次只作单方向移动，直到到达边缘磁道为止，然后再作反向移动。
- 下一次待访问的磁道只能在此磁头移动的前方，且选择磁头移动距离最近的一个磁盘请求响应。
- 又称为“电梯调度算法”。
- 消除了饥饿现象。

有九个进程先后提出磁盘I/O请求：
55, 58, 39, 18, 90,
160, 150, 38, 184

(从100号磁道开始, 向磁道号增加方向访问)	
被访问的下一个磁道号	移动距离(磁道数)
150	50
160	10
184	24
90	94
58	32
55	3
39	16
38	1
18	20
平均寻道长度: 27.8	

SCAN调度算法

2025/12/8/Mon

85

4、循环扫描（CScan）算法

- 磁头只作由内向外的单方向扫描，到达外边缘后，则返回最内侧的磁道重新进行下一轮扫描。
- 改进了对于边缘区磁道访问的不公平。

有九个进程先后提出
磁盘I/O请求：
55, 58, 39, 18, 90,
160, 150, 38, 184

(从100号磁道开始, 向磁道号增加方向访问)	
被访问的下一个磁道号	移动距离(磁道数)
150	50
160	10
184	24
18	166
38	20
39	1
55	16
58	3
90	32
平均寻道长度: 35.8	

CSCAN调度算法

2025/12/8/Mon

87

5、N步SCAN算法

- 磁臂粘着现象——某一磁道有较高的访问频率，进程反复请求对该磁道的I/O操作，从而垄断整个磁盘设备。
- N步SCAN算法：
 - 1、将磁盘请求队列分成若干个长度为N的子队列，磁盘调度将按FCFS算法依次处理这些子队列
 - 2、而每处理一个队列时，又是按SCAN算法，对一个队列处理完毕，再处理其他队列。
 - 3、当正在处理某子队列时，又有新的IO请求，将新请求进程放入其他队列。

有九个进程先后提出磁盘I/O请求：

55, 58, 39, 18, 90, 160, 150, 38, 184

例如，可以把这9个请求分成3个子队列：

队列一： 55、58、39

队列二： 18、90、160

队列三： 150、38、184

队列之间采用FCFS算法来处理。

每个队列内部的处理方式scan

6、FSCAN算法

- 该算法是N步SCAN算法的简化。
- 把磁盘请求队列分成两个子队列。
- 一个是当前所有的磁盘请求进程形成的队列。按SCAN算法进行处理。
- 在扫描期间，将新出现的所有请求进程，放入另一个等待处理的请求队列。
- 所有的新请求都被推迟到下一次扫描时处理。

5.6.3 磁盘高速缓存

- 磁盘I/O已经成为计算机系统的瓶颈。解决方法——磁盘高速缓存.
- 1、磁盘高速缓存的形式

第一种：在内存中开辟一个单独的存储空间作为磁盘缓存，大小固定，不受应用程序多少的影响。

第二种：把所有未利用的内存空间变成一个缓冲池，供请求分页系统和磁盘IO共享。此时的磁盘缓存大小不固定。应用程序较多时，缓冲池中的剩余内存空间就会较少。

- **2、数据交付方式**

数据交付：将磁盘高速缓存中的数据传送给请求者进程。

当有一进程请求访问某个盘块中的数据时，由内核先去查看高速缓存，是否存在所需盘块数据的拷贝。若有，则直接提取；否则，从磁盘中读入，同时送往高速缓存。

- **两种方式将数据交付给请求进程：**

(1) 数据交付。直接将高速缓存中的数据传至请求进程工作区。

(2) 指针交付。将指向高速缓存中某区域的指针交付请求进程。

- 3、置换算法
- 当高速缓存中已装满盘块数据，新的磁盘盘块数据无法装入时，就需要进行置换。
- 常用算法：最近最久未使用算法LRU
- 最近未使用算法NRU
- 最少使用算法LFU

- 4、周期性的写回磁盘

根据LRU算法，经常被访问的盘块数据，可能会一直保留在高速缓存中，长期不被写回磁盘。

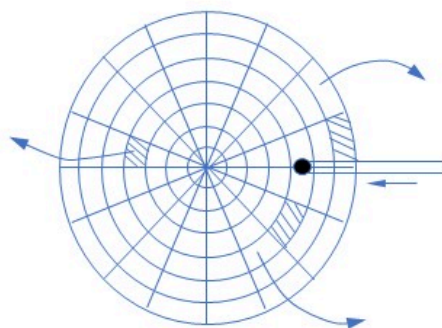
问题：若突然掉电，则信息全部丢失。

- **Unix的解决办法：**增设update程序，周期性的调用系统调用SYNC，强制性的将所有在高速缓存中已经修改的盘块数据写回磁盘。间隔30s。
- **MS-DOS的解决办法：**只要高速缓存中的某盘块数据被修改，则立即将它写回磁盘。优点：不会造成数据丢失，缺点：频繁启动磁盘。

主观题 10分

假设计算机系统采用CSCAN(循环扫描)磁盘调度策略，设某单面磁盘旋转速度为每分钟6000转，每个磁道有100个扇区，相邻磁道间的平均移动时间为1ms。

若在某时刻，磁头位于100号磁道处，并沿着磁道号增大的方向移动(如下图所示)，磁道号请求队列为50,90,30,120，对请求队列中的每一个磁道需读取1个随机分布的扇区，则读完这4个扇区总共需要多少时间？给出计算过程。



主观题 10分

如果将磁盘替换为随机访问的Flash半导体存储器(如U盘、SSD等), 是否有比CSCAN更高效的磁盘调度策略? 若有, 给出磁盘调度策略的名称并说明理由; 若无, 说明理由